

Time Is Space

Michael Timothy Bennett

Abstract

Complexity theory carves the world into time and space. Here I prove the distinction is an artefact of description, not of computation. I define a formalism in which representations are physically embodied and thus finite. In that setting, time is sequentially realised distinction and space is simultaneously realised distinction. Both are counts of a single underlying resource I call *tone*. Space is a chord of simultaneous tones. Time is an arpeggio of sequential tones. I prove two identities. Cumulative space through a bounded window is exactly ordinary space in a lifted vocabulary. Elapsed time is exactly a space count in a tick vocabulary. A bounded Turing-machine reconstruction recovers classical machine time and used-space as instances of this single resource. An exact block-product theorem further shows that literal-count simplicity and weakness (the count of compatible completions) coincide only when representational blocks are perfectly uniform. Real machines almost never satisfy this condition. The standard complexity classes are not affected. But time and space are not carved into computation. You carved them in when you chose a vocabulary.

Keywords: computational complexity, philosophy of computation, abstraction, time complexity, space complexity, levels of description

1 Introduction

The distinction between time and space is so entrenched that complexity theory is organised around it (Papadimitriou, 1994; Sipser, 2012). Hierarchy theorems, NP-completeness (Cook, 1971; Karp, 1972), and classical tradeoff results like Savitch’s theorem (Savitch, 1970) and the Hopcroft-Paul-Valiant simulation (Hopcroft et al., 1977) all presuppose that time and space are two different kinds of computational resource. I prove that they are not. At a finite abstraction layer, a bounded system can realise only finitely many distinctions. When those distinctions are realised sequentially, the cost appears as time. When they are realised simultaneously, the cost appears as space. There is one resource here, not two. Each distinction a vocabulary draws at a state is a *tone*. When tones are realised simultaneously, they form a chord, and the count is space. When they are realised sequentially, they form an arpeggio, and the count is time. The terms “chord” and “arpeggio” are borrowed from earlier work on the temporal structure of consciousness (Bennett, 2026). There, the question was whether the ingredients of a conscious moment must be co-instantiated (chord) or merely occur within a temporal window (arpeggio). Here the same distinction appears in a different guise. The difference between chord and arpeggio is the difference between spatial and temporal cost.

Within my formalism, I prove time and space amount to the same thing by proving two identities. *Cumulative* layer space along a bounded window is really *ordinary* layer space in a lifted vocabulary (Theorem 4.3). Elapsed layer time within that window is really a layer-space count in a tick vocabulary (Proposition 4.5). These identities follow from a state-based formalism in which environments are sets of mutually exclusive states (each state being a change or difference rather than possessing any ontological or semantic content), vocabularies are finite, and time is defined by change in encoding (Bennett, 2024a, 2026, 2025b). A bounded Turing-machine reconstruction then recovers classical machine time and machine space as instances of the framework’s single resource. An exact block-product theorem (Theorem 5.3) further shows that weakness (the volume of compatible completions) and literal-count simplicity coincide only when all representational blocks have the same size, a condition bounded machine vocabularies almost never satisfy (Bennett, 2024b, 2025a).

Dennett (1991) argued that which patterns count as real depends on the vocabulary used to describe them. Ladyman and Ross (2007) generalised this into a structural realism in which no description level is metaphysically privileged (French, 2014; Floridi, 2011). These ideas have informed work on levels of description in computation (Marr, 1982; Piccinini, 2015; Shagrir, 2006; Floridi, 2008; Newell, 1982). Separately, Blum (1967) showed that complexity measures can be studied abstractly, independent of any machine model, and Aaronson (2013) argued that philosophers should care about complexity because it constrains what is feasible, not merely what is computable. Putnam (1988) went further, arguing that every open system trivially implements every finite-state automaton, so computational descriptions are radically underdetermined without external constraints. The present result pushes all of this one step further. The very distinction between temporal and spatial cost is itself vocabulary-relative. Fix a different vocabulary, and the distinction changes. In the limit where every truth set is primitive, the distinction loses its bite. What the present argument challenges is the meta-assumption that the time/space distinction is a fact about computation rather than a fact about how you chose to look at it.

2 Vocabulary and Language

The definitions below continue a line of earlier work on finite abstraction layers, embodied computation, and computational dualism (Bennett, 2024a, 2025b, 2026). The standard formalisms of Turing (1936), Church (1936), and Post (1936) define computation in terms of mechanical procedures over symbols. The present formalism instead takes states and vocabularies as primitive, so that the representational resources of a bounded system are explicit in the definitions.

Definition 2.1 (Environment and programs). *An environment is a nonempty set Φ of mutually exclusive states. A program is a set $p \subseteq \Phi$. A program p is true at state ϕ iff $\phi \in p$. Let $P := 2^\Phi$ be the set of all programs.*

Put plainly, the environment is everything that could be the case. A program picks out which cases it is true in. Conjunction is intersection.

Definition 2.2 (Vocabulary and language). *A vocabulary is a finite nonempty set $\mathbf{v} \subseteq P$.*

The language generated by $\mathbf{v}$ is

$$L_{\mathbf{v}} := \left\{ l \subseteq \mathbf{v} : l \text{ is finite and } \bigcap_{p \in l} p \neq \emptyset \right\}.$$

Elements $l \in L_{\mathbf{v}}$ are called statements. A statement l is true at ϕ iff $\phi \in \bigcap_{p \in l} p =: T(l)$. In particular, $T(\emptyset) = \Phi$.

The vocabulary is the set of primitives you can use. The language is the set of consistent conjunctions of those primitives. A statement is satisfiable if some state makes all its components true. Throughout this paper I assume vocabularies are *non-degenerate*, meaning every $p \in \mathbf{v}$ satisfies $\emptyset \subsetneq p \subsetneq \Phi$. A program true at every state or false at every state carries no information and has no place in a vocabulary.

Definition 2.3 (Extension). *The extension of a statement $l \in L_{\mathbf{v}}$ is*

$$\text{Ext}(l) := \{l' \in L_{\mathbf{v}} : l \subseteq l'\}.$$

In other words, the extension is all the ways you can still complete the statement while staying consistent.

Definition 2.4 (Weakness and description orderings). *For statements $l \in L_{\mathbf{v}}$, define the extension count*

$$w(l) := |\text{Ext}(l)|$$

and the literal count

$$d(l) := |l|.$$

For statements $l_1, l_2 \in L_{\mathbf{v}}$, write

1. $l_1 <_w l_2$ iff $w(l_1) < w(l_2)$,
2. $l_1 <_d l_2$ iff $d(l_1) > d(l_2)$.

More completions mean greater weakness. Fewer literals mean greater simplicity. I write $l_1 <_w l_2$ when l_1 has fewer extensions than l_2 , so that moving downward in either order corresponds to adding detail.

3 Time and Space as Distinction Counts

Definition 3.1 (State sequence and objective trajectory). *A state sequence is any function $\tau : \mathbb{N} \rightarrow \Phi$. A state sequence is an objective trajectory if $\tau(t+1) \neq \tau(t)$ for all $t \in \mathbb{N}$.*

A state sequence is any history of states, including ones that repeat. An objective trajectory is a state sequence in which every step is a genuine change. Bounded machine runs that self-loop after halting are state sequences but not objective trajectories.

Definition 3.2 (Encoding map). *For a vocabulary $\mathbf{v} \subseteq P$ and state $\phi \in \Phi$, define $\text{Enc}_{\mathbf{v}}(\phi) := \{p \in \mathbf{v} : \phi \in p\}$.*

The encoding collects all programs in the vocabulary that are true at that state.

Definition 3.3 (Layer time). *Given a state sequence τ and vocabulary $\mathbf{v}$, the induced layer trajectory is $\tau^{(\mathbf{v})}(t) := \text{Enc}_{\mathbf{v}}(\tau(t))$.*

Define the layer-tick map $\kappa_{\mathbf{v}} : \mathbb{N} \rightarrow \mathbb{N}$ by $\kappa_{\mathbf{v}}(0) = 0$ and for $t > 0$:

$$\kappa_{\mathbf{v}}(t) := \begin{cases} \kappa_{\mathbf{v}}(t-1) & \text{if } \tau^{(\mathbf{v})}(t) = \tau^{(\mathbf{v})}(t-1), \\ \kappa_{\mathbf{v}}(t-1) + 1 & \text{otherwise.} \end{cases}$$

The layer time counts how many times the encoding has changed.

Layer time advances only when the encoding changes. If the vocabulary cannot distinguish two consecutive states, no time passes at that layer. For an objective trajectory, every objective step produces at least one vocabulary for which the encoding changes. For a state sequence that self-loops, the layer clock stops at every vocabulary once the loop begins. Earlier work defines the full compressed layer trajectory and the induced layer-time set (Bennett, 2026). Here I use only the counting fragment, because the present theorems depend on whether the layer clock ticks rather than on the full trajectory.

Proposition 3.4 (Monotonicity of layer time). *Let $\mathbf{v}' \subseteq \mathbf{v}$. Then $\kappa_{\mathbf{v}'}(t) \leq \kappa_{\mathbf{v}}(t)$ for all $t \in \mathbb{N}$.*

Proof. It suffices to show that every tick at $\mathbf{v}'$ implies a tick at $\mathbf{v}$. Suppose $\text{Enc}_{\mathbf{v}'}(\tau(t)) \neq \text{Enc}_{\mathbf{v}'}(\tau(t-1))$. Then some $p \in \mathbf{v}'$ is true at one of $\tau(t), \tau(t-1)$ and false at the other. Since $\mathbf{v}' \subseteq \mathbf{v}$, this p is also in $\mathbf{v}$, so $\text{Enc}_{\mathbf{v}}(\tau(t)) \neq \text{Enc}_{\mathbf{v}}(\tau(t-1))$. Every coarse tick forces a fine tick. Summing gives the inequality. $\square$

Abstraction can only slow time down, never speed it up. Removing distinctions from the vocabulary can only reduce the number of ticks, because fewer primitives means fewer opportunities for the encoding to change. A system described at a coarser level of abstraction cannot have a faster subjective clock than the same system described at a finer level. The converse does not circumvent this. Expanding the vocabulary below a given level can pack more content into each surviving tick at that level, but only by absorbing distinctions that were previously accounted for temporally. The total resource is redistributed, not reduced (Remark 4.8).

Definition 3.5 (Layer space). *The layer space of a vocabulary $\mathbf{v}$ at state ϕ is the cardinality $|\text{Enc}_{\mathbf{v}}(\phi)|$.*

In other words, layer space is how many distinctions the vocabulary instantiates simultaneously at a given state. This is a local occupancy count, not the vocabulary-level encoding capacity $l(\mathbf{v}) = \log_2 |\text{Im}(\text{Enc}_{\mathbf{v}})|$ defined in earlier work (Bennett, 2024a). The logarithmic form follows the standard convention for measuring information (Shannon, 1948). The encoding capacity is a ceiling on how many states the vocabulary can tell apart. Layer space counts how many primitives are active at one state.

Proposition 3.6 (Bound on layer space). *$|\text{Enc}_{\mathbf{v}}(\phi)| \leq |\mathbf{v}|$ for all $\phi \in \Phi$.*

Proof. $\text{Enc}_{\mathbf{v}}(\phi) \subseteq \mathbf{v}$. $\square$

Layer time counts encoding-changes. Layer space counts simultaneously active primitives. Both count tones. Layer space is the size of the chord at a state. Layer time is the length of the arpeggio through a window. The next section shows that this difference is not fundamental.

4 The Unification

The state ontology extends naturally to whole computations. The right move is to lift bounded histories into derived state spaces.

Definition 4.1 (Window environment and temporal lifts). *Fix a finite horizon $\Delta \in \mathbb{N}$. Define the window environment*

$$\Phi^{[\Delta]} := \Phi^{\Delta+1}.$$

An element $\sigma \in \Phi^{[\Delta]}$ is a tuple $\sigma = (\phi_0, \dots, \phi_\Delta)$. For each program $p \subseteq \Phi$ and each $t \in \{0, \dots, \Delta\}$, define the time-indexed lift

$$p@t := \{\sigma \in \Phi^{[\Delta]} : \phi_t \in p\}.$$

Given a vocabulary $\mathbf{v} \subseteq P$, define its lifted vocabulary

$$\mathbf{v}^{[\Delta]} := \{p@t : p \in \mathbf{v}, 0 \leq t \leq \Delta\}.$$

In other words, a window is itself a state in a derived environment. A time-indexed lift turns “ p is true at time t ” into a property of whole windows.

Remark 4.2 (Relation to earlier notation). *Earlier work (Bennett, 2026) defines existential and universal temporal lifts $\Diamond_\Delta p$ and $\Box_\Delta p$. The position-indexed lift $p@t$ is finer-grained. The two are related by*

$$\Diamond_\Delta p = \bigcup_{t=0}^{\Delta} (p@t), \quad \Box_\Delta p = \bigcap_{t=0}^{\Delta} (p@t).$$

The present counting arguments need exact time indices, not “somewhere” or “always” operators. In words, $\Diamond_\Delta p$ is true at a window if p holds at some time step, and $\Box_\Delta p$ requires p at every time step.

Theorem 4.3 (Cumulative layer space is layer space in a lifted vocabulary). *Let $\mathbf{v}$ be a finite vocabulary and let $\sigma = (\phi_0, \dots, \phi_\Delta) \in \Phi^{[\Delta]}$. Then*

$$|\text{Enc}_{\mathbf{v}^{[\Delta]}}(\sigma)| = \sum_{t=0}^{\Delta} |\text{Enc}_{\mathbf{v}}(\phi_t)|.$$

Proof. By definition,

$$\text{Enc}_{\mathbf{v}^{[\Delta]}}(\sigma) = \{p@t \in \mathbf{v}^{[\Delta]} : \sigma \in p@t\}.$$

But $\sigma \in p@t$ iff $\phi_t \in p$, so

$$\text{Enc}_{\mathbf{v}^{[\Delta]}}(\sigma) = \bigcup_{t=0}^{\Delta} \{p@t : p \in \text{Enc}_{\mathbf{v}}(\phi_t)\}.$$

The sets on the right are pairwise disjoint. To see this, suppose $p@t_1 = q@t_2$ as subsets of $\Phi^{[\Delta]}$ with $t_1 \neq t_2$. By non-degeneracy there exists $b \notin q$, and there exists $a \in p$. The window σ with $\phi_{t_1} = a$ and $\phi_{t_2} = b$ satisfies $\sigma \in p@t_1$ but $\sigma \notin q@t_2$, a contradiction. (The coordinates ϕ_{t_1} and ϕ_{t_2} are independent in $\Phi^{\Delta+1}$.) Taking cardinalities gives the identity. $\square$

So now what looked like a temporal accumulation of spatial distinctions is just one simultaneous distinction count in the lifted state space. Note that this theorem concerns cumulative layer space (a sum through time), not peak layer space at a single instant. Classical space complexity is usually a peak measure. The connection to peak space comes through Proposition 6.6, which recovers instantaneous used-space directly as layer space at a single state. The present theorem makes a different and complementary point, which is that the *temporal* accumulation of spatial cost is itself a spatial quantity once the window is treated as a state.

Definition 4.4 (Tick vocabulary). *Fix $\mathbf{v}$ and $\Delta \geq 1$. For each $t \in \{1, \dots, \Delta\}$ define the tick program*

$$\text{Tick}_t^{\mathbf{v}} := \{\sigma = (\phi_0, \dots, \phi_\Delta) \in \Phi^{[\Delta]} : \text{Enc}_{\mathbf{v}}(\phi_t) \neq \text{Enc}_{\mathbf{v}}(\phi_{t-1})\}.$$

Let

$$\Theta_{\mathbf{v}}^{[\Delta]} := \{\text{Tick}_1^{\mathbf{v}}, \dots, \text{Tick}_\Delta^{\mathbf{v}}\}.$$

Proposition 4.5 (Elapsed layer time is layer space in the tick vocabulary). *For every window $\sigma = (\phi_0, \dots, \phi_\Delta) \in \Phi^{[\Delta]}$,*

$$|\text{Enc}_{\Theta_{\mathbf{v}}^{[\Delta]}}(\sigma)| = |\{t \in \{1, \dots, \Delta\} : \text{Enc}_{\mathbf{v}}(\phi_t) \neq \text{Enc}_{\mathbf{v}}(\phi_{t-1})\}|.$$

Proof. $\text{Tick}_t^{\mathbf{v}}$ is true at σ iff the $\mathbf{v}$ -encoding changes between times $t - 1$ and t . So the true programs in $\Theta_{\mathbf{v}}^{[\Delta]}$ are exactly those indices at which the layer clock ticks. Counting them gives the identity. $\square$

Hence the name of this paper. An arpeggio in one vocabulary is a chord in a lifted one. Time is space.

Example 4.6 (The unification in miniature). *Let $\Phi = \{1, 2, 3, 4\}$, and let $\mathbf{v} = \{p, q\}$ where $p = \{1, 2\}$ and $q = \{2, 3\}$. Consider a window of length $\Delta = 2$ with $\sigma = (\phi_0, \phi_1, \phi_2) = (1, 2, 3)$. At $\phi_0 = 1$, only p is true, so $|\text{Enc}_{\mathbf{v}}(\phi_0)| = 1$. At $\phi_1 = 2$, both p and q are true, so $|\text{Enc}_{\mathbf{v}}(\phi_1)| = 2$. At $\phi_2 = 3$, only q is true, so $|\text{Enc}_{\mathbf{v}}(\phi_2)| = 1$. Cumulative layer space through the window is $1 + 2 + 1 = 4$.*

The lifted vocabulary $\mathbf{v}^{[2]} = \{p@0, p@1, p@2, q@0, q@1, q@2\}$ has six elements. At σ , the true lifted programs are $p@0$, $p@1$, $q@1$, and $q@2$. That is four simultaneously true programs. So $|\text{Enc}_{\mathbf{v}^{[2]}}(\sigma)| = 4$, confirming the identity. Temporal accumulation of distinctions has become simultaneous distinction in the lifted vocabulary.

Now consider elapsed layer time. The encoding changes between ϕ_0 and ϕ_1 (from $\{p\}$ to $\{p, q\}$), and again between ϕ_1 and ϕ_2 (from $\{p, q\}$ to $\{q\}$). So the layer clock ticks twice. The tick vocabulary $\Theta_{\mathbf{v}}^{[2]} = \{\text{Tick}_1, \text{Tick}_2\}$ has both programs true at σ , giving $|\text{Enc}_{\Theta_{\mathbf{v}}^{[2]}}(\sigma)| = 2$. Elapsed time is a space count in the tick vocabulary.

Remark 4.7 (Non-triviality). *One might object that embedding a temporal sequence into a product space trivially converts any temporal count into a spatial count. The objection misidentifies what is happening. The window lift is not an arbitrary re-encoding chosen to produce the desired conclusion. It is the formalism's canonical method for representing bounded histories within its own state-based ontology (Bennett, 2026). The formalism takes instantaneous states as primitive. When it needs to talk about sequences of states, it does so by*

constructing derived environments whose states are windows. This is the only move available that respects the ontology. The philosophical discovery is that making this move automatically unifies time and space. The unification is forced by the formalism, not manufactured by the analyst.

Remark 4.8 (Why this does not collapse complexity classes). *It is natural to ask whether converting temporal cost into spatial cost collapses P into $PSPACE$ or settles P versus NP . It does not, for two reasons that together entail conservation of total resource.*

First, monotonicity (Proposition 3.4). Coarsening the vocabulary cannot create ticks. You can only lose temporal resolution by removing distinctions, never gain it. So no vocabulary change can manufacture a faster computation from a slower one.

Second, conservation. Expanding the vocabulary below a given level can enrich what happens at each tick, but only by absorbing exactly the distinctions that were previously counted temporally. The window lift makes this explicit. A T -step computation over vocabulary $\mathbf{v}$ lifts to a vocabulary of $|\mathbf{v}| \cdot (T + 1)$ elements. All the temporal ticks become simultaneous distinctions. The tick count at the lifted level drops. But the chord at the lifted level has absorbed everything the arpeggio contained.

Together, vocabulary change redistributes resource between time and space. It cannot reduce the total. Complexity classes ask whether there exist machines that solve certain problems within certain bounds. The present result is about how you describe the cost of a machine that already exists. Changing vocabulary does not change what the machine does. It changes how you account for what the machine does.

Remark 4.9 (Why abstraction matters). *These results are nontrivial because the vocabulary is a sparse subset of P , not because it is merely finite and non-degenerate. To see why sparsity matters, consider the limiting case where Φ is finite and $\mathbf{v} = 2^\Phi \setminus \{\emptyset, \Phi\}$, which is finite and non-degenerate. In this vocabulary, every nonempty proper truth set is already a primitive, so every such truth set has a one-literal representative. Multi-literal statements still exist, but no truth set requires more than one literal to name. The scarcity of representational resources has disappeared even though the vocabulary is finite and non-degenerate. Layer time and layer space are still well-defined. The time/space distinction does not vanish. What vanishes is expressive scarcity, the gap between what exists and what the vocabulary can express (Bennett, 2024a,b). It is that gap which makes complexity distinctions substantive. In any realistic setting, the vocabulary is vastly smaller than P , and this gap is what gives the framework its content.*

5 Weakness and Simplicity in Product Vocabularies

Once time and space are recognised as projections of a single resource, the question becomes how to measure that resource. The obvious candidate is literal count, which measures description length. Description length has a long history as a complexity measure (Kolmogorov, 1965; Chaitin, 1966; Solomonoff, 1964), and the minimum description length principle treats it as the foundation of statistical inference (Rissanen, 1978; Grünwald, 2007). But literal count is encoding-dependent. A natural question is when it agrees with extension count,

which measures weakness and is encoding-independent. Earlier work showed that the two come apart in general (Bennett, 2024b, 2025a). Here I give the exact criterion.

Definition 5.1 (Partition-product vocabulary). *Let $\Phi = \prod_{i=1}^m A_i$ with each A_i finite and $|A_i| \geq 2$. For each coordinate i and value $a \in A_i$, define the coordinate program $p_{i,a} := \{x \in \Phi : x_i = a\}$. Let $\mathbf{v}_{\text{prod}} := \{p_{i,a} : 1 \leq i \leq m, a \in A_i\}$. For each coordinate define the block $B_i := \{p_{i,a} : a \in A_i\}$. The condition $|A_i| \geq 2$ ensures that every $p_{i,a}$ is non-degenerate.*

Lemma 5.2 (Product extension count). *Every statement $l \in L_{\mathbf{v}_{\text{prod}}}$ fixes at most one value in each block. Let $I(l) \subseteq \{1, \dots, m\}$ be the set of coordinates fixed by l . Then*

$$|\text{Ext}(l)| = \prod_{i \notin I(l)} (|A_i| + 1).$$

Proof. A statement is satisfiable iff it chooses at most one literal from each block B_i . A completion of l is obtained by, independently for each unfixed coordinate $i \notin I(l)$, either adding no new literal from B_i or exactly one literal from B_i . The “+1” accounts for leaving a coordinate unfixed. When no new coordinate is fixed at all, the resulting completion is l itself. The choices multiply across coordinates. $\square$

Theorem 5.3 (Exact criterion for weakness-simplicity alignment). *For the vocabulary $\mathbf{v}_{\text{prod}}$, the orderings $<_w$ and $<_d$ coincide on $L_{\mathbf{v}_{\text{prod}}}$ if and only if*

$$|A_1| = |A_2| = \dots = |A_m|.$$

Proof. If all block sizes equal some d , then by Lemma 5.2,

$$|\text{Ext}(l)| = (d + 1)^{m - |l|},$$

which is strictly decreasing in $|l|$. So $<_w$ and $<_d$ coincide. Conversely, suppose the orderings coincide. For each i , choose any $a_i \in A_i$ and consider the singleton $\{p_{i,a_i}\}$. All singletons have description length 1, so none are comparable under $<_d$. Therefore none can be comparable under $<_w$. By Lemma 5.2,

$$|\text{Ext}(\{p_{i,a_i}\})| = \prod_{k \neq i} (|A_k| + 1).$$

Equal for all i forces $|A_i| = |A_j|$ for all i, j . $\square$

Put another way, simplicity and weakness agree only when the representational building blocks are perfectly uniform. The moment the blocks differ in size, two statements can have the same length but different weakness values.

6 Bounded Turing Machines

The framework takes vocabularies to be finite. To connect with classical complexity theory I fix an input length and a space envelope, and represent the machine by bounded instantaneous states (Turing, 1936; Sipser, 2012; Hopcroft et al., 2001). The choice of machine model matters. Simulating k tapes on two tapes costs a logarithmic overhead in time (Hennie and

(Stearns, 1966), and the relationship between model choice and complexity measurement has been studied extensively (van Emde Boas, 1990; Goldreich, 2008). Throughout this section, Turing machines have a one-way semi-infinite tape indexed by $1, 2, \dots$, with the head initially at cell 1. I assume $|Q| \geq 2$, $S \geq 2$, and $|\Gamma| \geq 2$. These conditions ensure that all coordinate programs defined below are non-degenerate. They are satisfied by any machine and bound of practical interest.

Definition 6.1 (Bounded instantaneous TM environment). *Fix a deterministic Turing machine $M = (Q, \Sigma, \Gamma, \sqcup, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$ where $\Sigma \subseteq \Gamma$ is the input alphabet, $\sqcup \in \Gamma \setminus \Sigma$ is the blank symbol, q_0 , q_{acc} , and q_{rej} are pairwise distinct, and δ is total on non-halting states (that is, $\delta(q, a)$ is defined for all $q \notin \{q_{\text{acc}}, q_{\text{rej}}\}$ and all $a \in \Gamma$). Fix a space bound $S \in \mathbb{N}_{>0}$ with $|Q| \geq 2$, $S \geq 2$, $|\Gamma| \geq 2$. Define the bounded instantaneous environment*

$$\Phi_{M,S} := Q \times \{1, \dots, S\} \times \Gamma^S \times \{0, 1\}^S.$$

An element $\phi = (q, h, T, u)$ consists of a control state q , a head position h , a bounded tape content function T , and a usage marker u where $u_j = 1$ means cell j has been visited.

Definition 6.2 (Bounded TM vocabularies). *On $\Phi_{M,S}$ define the following programs.*

$$\begin{aligned} Q_q &:= \{(q', h, T, u) \in \Phi_{M,S} : q' = q\} & (q \in Q), \\ H_i &:= \{(q, h, T, u) \in \Phi_{M,S} : h = i\} & (1 \leq i \leq S), \\ C_{j,a} &:= \{(q, h, T, u) \in \Phi_{M,S} : T(j) = a\} & (1 \leq j \leq S, a \in \Gamma), \\ U_j &:= \{(q, h, T, u) \in \Phi_{M,S} : u_j = 1\} & (1 \leq j \leq S). \end{aligned}$$

Let $\mathbf{v}_{M,S}^{\text{conf}} := \{Q_q : q \in Q\} \cup \{H_i : 1 \leq i \leq S\} \cup \{C_{j,a} : 1 \leq j \leq S, a \in \Gamma\}$ and $\mathbf{v}_{M,S}^{\text{use}} := \{U_j : 1 \leq j \leq S\}$. For the initial-state specification (Section 6.1), complement usage programs are also needed. Define $\bar{U}_j := \{(q, h, T, u) \in \Phi_{M,S} : u_j = 0\}$ for $1 \leq j \leq S$. Let $\bar{\mathbf{v}}_{M,S}^{\text{use}} := \{\bar{U}_j : 1 \leq j \leq S\}$.

Definition 6.3 (Bounded transition relation). *Let $\phi = (q, h, T, u)$ and $\phi' = (q', h', T', u')$ be states in $\Phi_{M,S}$. Write $\phi \Rightarrow_{M,S} \phi'$ iff one of the following holds:*

1. $q \in \{q_{\text{acc}}, q_{\text{rej}}\}$ and $\phi' = \phi$ (halting states self-loop),
2. $q \notin \{q_{\text{acc}}, q_{\text{rej}}\}$ and all of the following hold: $\delta(q, T(h)) = (q', a, D)$ for some $a \in \Gamma$ and move $D \in \{L, R, N\}$, T' agrees with T except possibly at h and $T'(h) = a$, $h' = h - 1$, $h' = h + 1$, or $h' = h$ according as $D = L$, $D = R$, or $D = N$ and $1 \leq h' \leq S$, and for each $1 \leq j \leq S$ one has $u'_j = 1$ iff either $u_j = 1$ or $j = h'$.

A legal bounded run of M inside $\Phi_{M,S}$ is a sequence $\tau = (\phi_0, \phi_1, \dots)$ with $\phi_t \Rightarrow_{M,S} \phi_{t+1}$ for every t . The self-loop convention ensures that legal runs are defined for all t , including times after the machine has halted.

Proposition 6.4 (TM time is layer time). *Assume M is in non-stuttering form, meaning each legal transition from a non-halting state changes at least one of the control state, head position, or tape contents. Then each non-halting machine step contributes exactly one layer-time tick in $\mathbf{v}_{M,S}^{\text{conf}}$, and halting self-loops contribute zero ticks.*

Proof. A legal non-halting step changes at least one of (q, h, T) . The vocabulary $\mathbf{v}_{M,S}^{\text{conf}}$ distinguishes all three. So $\text{Enc}_{\mathbf{v}_{M,S}^{\text{conf}}}(\phi_{t+1}) \neq \text{Enc}_{\mathbf{v}_{M,S}^{\text{conf}}}(\phi_t)$ for every non-halting transition, and the layer clock ticks once per such step. A halting self-loop leaves the entire configuration unchanged, so $\text{Enc}_{\mathbf{v}_{M,S}^{\text{conf}}}(\phi_{t+1}) = \text{Enc}_{\mathbf{v}_{M,S}^{\text{conf}}}(\phi_t)$ and the layer clock does not tick. $\square$

Remark 6.5 (Non-stuttering). *The non-stuttering assumption requires that each legal transition from a non-halting state changes at least one of (q, h, T) . It is stated relative to these coordinates because $\mathbf{v}_{M,S}^{\text{conf}}$ does not distinguish states that differ only in usage markers. Halting states are excluded because the self-loop convention makes them stutter by design. The assumption is harmless. The bounded configuration graph induced by $\Rightarrow_{M,S}$ is finite, so any non-halting loop can be detected and replaced by a transition to q_{rej} . This preserves the accepted language and resource consumption on all inputs that halt within the bound (Sipser, 2012).*

Proposition 6.6 (TM space is layer space). *Define the used-space at time t by $\text{Space}_M(t) := |\{j : u_t(j) = 1\}|$. Then $\text{Space}_M(t) = |\text{Enc}_{\mathbf{v}_{M,S}^{\text{use}}}(\phi_t)|$.*

Proof. U_j is true at ϕ_t iff $u_t(j) = 1$. Taking cardinalities gives the identity. $\square$

Standard TM time and standard TM space are both tone counts in finite vocabularies. Neither is primitive. Both are projections of how a vocabulary cuts up the state space.

Corollary 6.7 (Weakness and simplicity diverge in bounded TM vocabularies). *If $|Q|$, S , and $|\Gamma|$ are not all equal, then $<_w$ and $<_d$ do not coincide on $L_{\mathbf{v}_{M,S}^{\text{conf}}}$.*

Proof. The configuration vocabulary $\mathbf{v}_{M,S}^{\text{conf}}$ ignores the usage markers, so the induced language is isomorphic to a product language on $Q \times \{1, \dots, S\} \times \Gamma^S$ with block sizes $|Q|$, S , and $|\Gamma|, \dots, |\Gamma|$. Apply Theorem 5.3. $\square$

Example 6.8 (Divergence in a concrete machine). *Take a machine with $|Q| = 5$ states, tape alphabet $\Gamma = \{0, 1\}$, and space bound $S = 4$. The three singleton extension counts are:*

$$\begin{aligned} |\text{Ext}(\{Q_q\})| &= (4+1)(2+1)^4 = 405, \\ |\text{Ext}(\{H_i\})| &= (5+1)(2+1)^4 = 486, \\ |\text{Ext}(\{C_{j,a}\})| &= (5+1)(4+1)(2+1)^3 = 810. \end{aligned}$$

All three are singletons, so they have the same description length. But specifying a tape symbol (810 remaining completions) leaves open twice as many options as specifying a control state (405 remaining completions). The literal count sees no difference. The weakness ordering sees a factor-of-two gap.

6.1 Computations as window states

A complete computation is not an instantaneous state. In the present ontology it is a window-state satisfying suitable window programs.

Definition 6.9 (Window programs for accepting runs). Fix M , S , and $\Delta \geq 1$. Work in the window environment $\Phi_{M,S}^{[\Delta]} := (\Phi_{M,S})^{\Delta+1}$. For each t with $0 \leq t < \Delta$, define

$$\text{Step}_t^M := \{\sigma = (\phi_0, \dots, \phi_\Delta) : \phi_t \Rightarrow_{M,S} \phi_{t+1}\},$$

and define

$$\text{Acc}^{[\Delta]} := \{\sigma = (\phi_0, \dots, \phi_\Delta) : \exists t \leq \Delta \text{ such that } \phi_t \text{ is accepting}\}.$$

Let $\mathfrak{w}_{M,S}^{[\Delta]} := (\mathfrak{v}_{M,S}^{\text{conf}} \cup \mathfrak{v}_{M,S}^{\text{use}} \cup \bar{\mathfrak{v}}_{M,S}^{\text{use}})^{[\Delta]} \cup \{\text{Step}_t^M : 0 \leq t < \Delta\} \cup \{\text{Acc}^{[\Delta]}\}.$

Definition 6.10 (Accepting-run conjunction). Fix an input string x over an input alphabet $\Sigma \subseteq \Gamma$ with $|x| \leq S$. The initial-state specification ι_x is the conjunction over $(\mathfrak{v}_{M,S}^{\text{conf}} \cup \mathfrak{v}_{M,S}^{\text{use}} \cup \bar{\mathfrak{v}}_{M,S}^{\text{use}})^{[\Delta]}$ that fixes the time-0 configuration to the standard bounded initial state for x . Explicitly, ι_x contains: the initial control state $Q_{q_0} @ 0$, the initial head position $H_1 @ 0$, the tape contents $C_{j,x_j} @ 0$ for $1 \leq j \leq |x|$ and $C_{j,\sqcup} @ 0$ for $|x| < j \leq S$ (where $\sqcup \in \Gamma$ is the blank symbol), and the initial usage markers $U_1 @ 0$ (the starting head position is the only visited cell) and $\bar{U}_j @ 0$ for $2 \leq j \leq S$. The accepting-run conjunction for M , x , S , and Δ is

$$R_{M,x,S,\Delta} := \iota_x \cup \{\text{Step}_0^M, \dots, \text{Step}_{\Delta-1}^M, \text{Acc}^{[\Delta]}\}.$$

This conjunction is a statement (satisfiable) when M accepts x within Δ steps using at most S tape cells. Otherwise it is unsatisfiable and lies outside $L_{\mathfrak{w}_{M,S}^{[\Delta]}}$.

Proposition 6.11 (Window semantics of accepting computation). The conjunction $R_{M,x,S,\Delta}$ is satisfiable if and only if M accepts x within Δ steps using at most S tape cells. When satisfiable, a window $\sigma \in \Phi_{M,S}^{[\Delta]}$ satisfies $R_{M,x,S,\Delta}$ iff σ is a legal Δ -step bounded run of M on x that reaches an accepting state within the window.

Proof. ι_x uniquely fixes the initial configuration at time 0, including the head position, tape contents, and usage markers. Each Step_t^M enforces one legal transition. After the machine enters a halting state, the self-loop convention (Definition 6.3) ensures that subsequent Step constraints are satisfied by the repeated halting configuration. $\text{Acc}^{[\Delta]}$ requires acceptance at some time in the window. The conjunction is satisfied exactly by legal bounded runs that accept. If no such run exists, the conjunction is unsatisfiable. $\square$

This preserves the state ontology. A full computation is an ordinary statement in a derived window environment rather than a primitive object of a different kind.

7 Discussion

The bounded Turing-machine reconstruction closes the main argument. Propositions 6.4 and 6.6 show that layer time and layer space are not alternative definitions of computational time and space. They recover the standard notions under the bounded non-stuttering representation, expressed in vocabulary-relative terms. The unification in Section 4 therefore applies directly to what complexity theorists already mean by time and space. The block-product theorem adds a second consequence. Once time and space are recognised as projections of one resource, the question becomes how to measure that resource. Theorem 5.3 and Corollary 6.7 answer that literal count is the wrong measure and weakness is the right one.

7.1 Consequences for philosophy of computation

The result has direct consequences for the mechanistic account of computation (Piccinini, 2015; Piccinini and Maley, 2021). On that account, a computational system is a mechanism that processes vehicles according to rules. Computational steps (temporal cost) and computational storage (spatial cost) are features of the mechanism itself. The present result challenges this. If the step/storage distinction arises from the vocabulary used to describe the mechanism rather than from the mechanism’s intrinsic structure, then what counts as a step and what counts as storage is not a fact about the physical system. It is a fact about the abstraction layer at which the system is described (Floridi, 2008). Two descriptions of the same physical process, at different vocabulary layers, can disagree about how much of the computational cost is temporal and how much is spatial. Neither is wrong, but they are projections of the same underlying resource onto different vocabulary axes. This also bears on the question of when a physical system implements a computation at all (Chalmers, 1996; Putnam, 1988).

This connects to work by Shagrir (2006) on computational individuation. Shagrir (2006) argued that the brain is viewed as a computer because computational descriptions are individuated by functional organisation, and functional organisation is level-relative. The present result makes a version of this precise for the specific case of resource individuation. It is not just the computational identity of a system that depends on the level of description (Fodor, 1974; Kim, 1993; Polger and Shapiro, 2016). The decomposition of its resource expenditure into temporal and spatial components depends on the level of description too. Part of the answer to why we view the brain as a computer is that we have fixed a vocabulary. The time/space decomposition of the brain’s computational work is an artefact of that choice.

More broadly, this connects to the thesis of Dennett (1991) that which patterns count as real depends on the vocabulary used to describe them. Dennett was concerned with intentional states. The patterns he had in mind were beliefs, desires, and other propositional attitudes (Haugeland, 1998). Ladyman and Ross (2007) generalised this into a structural realism in which the question “which patterns are real?” has no vocabulary-independent answer (French, 2014; Ross, 2000). The present result applies this structural insight to a case Dennett did not consider. The time/space distinction in computational complexity is a pattern. It is a real pattern at a given abstraction layer. But it does not survive changes of vocabulary (Wallace, 2003; Floridi, 2011). If structural realism is right that reality is pattern-relative, then time and space in computation are real in the same sense that beliefs are real on Dennett’s account. They are features of a description that earns its keep by being useful, not features of the world that exist independent of how it is described.

7.2 Consequences for classical complexity theory

Fixing a machine model is not a dispensable convenience. It is what generates the time/space distinction in the first place. The foundational results of Hartmanis and Stearns (1965) and Cobham (1965) defined complexity relative to a fixed machine model. The hierarchy theorems and standard class separations (Cook, 1971; Karp, 1972; Papadimitriou, 1994; Arora and Barak, 2009) all inherit that model-dependence. Every textbook acknowledges this (Sipser, 2012; Goldreich, 2008). The sensitivity of time complexity to model choice is well documented (Hennie and Stearns, 1966; van Emde Boas, 1990). What the present framework adds is a

precise account of what model-fixing is doing. In the full-vocabulary limit, expressive scarcity vanishes and complexity distinctions lose their substantive content (Remark 4.9). In any realistic setting, the vocabulary is sparse, and the time/space decomposition depends on which sparse vocabulary you chose.

Savitch’s theorem (Savitch, 1970) is a useful point of contact. It shows that nondeterministic space is at most the square of deterministic space. The Hopcroft-Paul-Valiant theorem (Hopcroft et al., 1977) established that time t can always be simulated in space $O(t/\log t)$, recently improved to roughly $O(\sqrt{t})$ up to logarithmic factors by Williams (2025). These are tradeoffs within a fixed model. The present framework explains why such tradeoffs exist at all. Time and space are projections of one resource onto different vocabulary axes. Different models assign different weights to these projections. That is why tradeoff ratios are model-dependent. Rather than a new tradeoff theorem, this is a diagnosis of why tradeoff theorems take the form they do.

Non-determinism does not affect the unification. The core theorems (Theorem 4.3 and Proposition 4.5) operate on state sequences and vocabularies. They never appeal to determinism. A nondeterministic machine generates a branching tree of possible runs, but each run is still a state sequence in Φ , and the unification holds for each. Rabin and Scott (1959) showed that nondeterministic finite automata accept the same languages as deterministic ones. Immerman (1988) and Szelepcsényi (1988) independently proved that nondeterministic space is closed under complementation, a result with no known time analogue. The present framework does not need any of these reductions. Non-determinism changes which state sequences are possible. It does not change the vocabulary-relative character of the time/space distinction within any given run. The bounded Turing-machine reconstruction in Section 6 assumes determinism for expository convenience, not out of necessity.

The block-product theorem adds a further correction. Even in the bounded TM setting, weakness and literal-count simplicity coincide only when the representational blocks are uniform. Real machines violate this (Example 6.8). Simplicity does not explain generalisation, persistence, or computational efficiency in this framework. Simplicity is encoding-dependent and can be gamed. The right measure within any given vocabulary is weakness, meaning how many further commitments remain available while staying correct (Bennett, 2024b,c, 2025a). If a persisting object is simple, that is incidental unless it is also weakly constrained.

7.3 Open questions

The present TM reconstruction assumes determinism, but as noted above, the core theorems hold for any state sequence regardless of how it was generated. Nondeterminism is therefore not an open problem for this framework. However, probabilistic and quantum computation are genuinely open, because they enrich the ontology of states itself (probability distributions over runs, superposition vectors) rather than merely multiplying the set of possible runs. The relationship between the present finite-layer account and asymptotic complexity classes also remains to be formalised. The framework’s state-based ontology may connect to further debates about physical computation and the implementation relation (Piccinini, 2015; Landauer, 1961; Bennett, 1973; Fredkin and Toffoli, 1982). Wheeler (1990) proposed that physics is fundamentally informational, and Lloyd (2000) derived physical limits on computation from quantum mechanics. Whether the present vocabulary-relative account

of computational resources connects to these physical accounts is an open question. The relationship between time and space in physics is itself contested. General relativity unifies them into spacetime, but this unification has been challenged from multiple directions (Barbour, 1999; Smolin, 2013; Rovelli, 2018; Price, 1996). Whether the present computational result is connected to the physical debate, or merely analogous, is a question this paper does not answer but invites.

Declarations

Funding. No funding was received for this work.

Conflict of interest. The author declares no conflict of interest.

Data availability. No datasets were generated or analysed. All results are mathematical.

References

- Scott Aaronson. Why philosophers should care about computational complexity. In B. Jack Copeland, Carl J. Posy, and Oron Shagrir, editors, *Computability: Turing, Gödel, Church, and Beyond*, pages 261–327. MIT Press, 2013.
- Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009. ISBN 9780521424264. URL <https://www.cambridge.org/9780521424264>.
- Julian Barbour. *The End of Time: The Next Revolution in Our Understanding of the Universe*. Oxford University Press, Oxford, 1999.
- Charles H. Bennett. Logical reversibility of computation. *IBM Journal of Research and Development*, 17(6):525–532, 1973. doi: 10.1147/rd.176.0525.
- Michael Timothy Bennett. Computational dualism and objective superintelligence. In *17th International Conference on Artificial General Intelligence*, Lecture Notes in Computer Science. Springer, 2024a. doi: 10.1007/978-3-031-65572-2_3. URL https://doi.org/10.1007/978-3-031-65572-2_3.
- Michael Timothy Bennett. Is complexity an illusion? In *17th International Conference on Artificial General Intelligence*, Lecture Notes in Computer Science. Springer, 2024b. doi: 10.1007/978-3-031-65572-2_2. URL https://doi.org/10.1007/978-3-031-65572-2_2.
- Michael Timothy Bennett. Are biological systems more intelligent than artificial intelligence?, 2024c. URL <https://arxiv.org/abs/2405.02325>. In press, 2026, Philosophical Transactions of the Royal Society B: Biological Sciences. Special issue on Hybrid agencies: crossing borders between biological and artificial worlds.
- Michael Timothy Bennett. Optimal policy is weakest policy. In *Artificial General Intelligence*, volume 16057 of *Lecture Notes in Computer Science*, pages 43–56. Springer, 2025a. doi: 10.1007/978-3-032-00686-8_5. URL https://doi.org/10.1007/978-3-032-00686-8_5.

- Michael Timothy Bennett. *How To Build Conscious Machines*. PhD thesis, The Australian National University, 2025b. URL <https://hdl.handle.net/1885/733782452>.
- Michael Timothy Bennett. A mind cannot be smeared across time. In *Forthcoming in Proceedings of the AAAI 2026 Spring Symposium on Machine Consciousness: Integrating Theory, Technology, and Philosophy*, Burlingame, California, USA, April 2026. AAAI Press.
- Manuel Blum. A machine-independent theory of the complexity of recursive functions. *Journal of the ACM*, 14(2):322–336, 1967. doi: 10.1145/321386.321395.
- Gregory J. Chaitin. On the length of programs for computing finite binary sequences. *J. ACM*, 1966.
- David J. Chalmers. Does a rock implement every finite-state automaton? *Synthese*, 108(3): 309–333, 1996. doi: 10.1007/BF00413692.
- Alonzo Church. An unsolvable problem of elementary number theory. *American Journal of Mathematics*, 58(2):345–363, 1936. doi: 10.2307/2371045.
- Alan Cobham. The intrinsic computational difficulty of functions. In Yehoshua Bar-Hillel, editor, *Logic, Methodology and Philosophy of Science: Proceedings of the 1964 International Congress*, Studies in Logic and the Foundations of Mathematics, pages 24–30. North-Holland Publishing Company, Amsterdam, 1965. URL <https://philpapers.org/rec/COBTIC>.
- Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pages 151–158, 1971. doi: 10.1145/800157.805047. URL <https://doi.org/10.1145/800157.805047>.
- Daniel C. Dennett. Real patterns. *The Journal of Philosophy*, 88(1):27–51, 1991.
- Luciano Floridi. The method of levels of abstraction. *Minds and Machines*, 18(3):303–329, 2008. doi: 10.1007/s11023-008-9113-7.
- Luciano Floridi. A defence of informational structural realism. *Synthese*, 181(2):219–253, 2011. doi: 10.1007/s11229-010-9702-z.
- Jerry A. Fodor. Special sciences (or: The disunity of science as a working hypothesis). *Synthese*, 28(2):97–115, 1974. doi: 10.1007/BF00485230.
- Edward Fredkin and Tommaso Toffoli. Conservative logic. *International Journal of Theoretical Physics*, 21(3–4):219–253, 1982. doi: 10.1007/BF01857727.
- Steven French. *The Structure of the World: Metaphysics and Representation*. Oxford University Press, Oxford, 2014. doi: 10.1093/acprof:oso/9780199684847.001.0001.
- Oded Goldreich. *Computational Complexity: A Conceptual Perspective*. Cambridge University Press, Cambridge, 2008. doi: 10.1017/CBO9780511804106.
- Peter D. Grünwald. *The Minimum Description Length Principle*. MIT Press, Cambridge, MA, 2007.

- Juris Hartmanis and Richard E. Stearns. On the computational complexity of algorithms. *Transactions of the American Mathematical Society*, 117:285–306, 1965. doi: 10.1090/S0002-9947-1965-0170805-7. URL <https://doi.org/10.1090/S0002-9947-1965-0170805-7>.
- John Haugeland. *Having Thought: Essays in the Metaphysics of Mind*. Harvard University Press, Cambridge, MA, 1998.
- F. C. Hennie and R. E. Stearns. Two-tape simulation of multitape Turing machines. *Journal of the ACM*, 13(4):533–546, 1966. doi: 10.1145/321356.321362.
- John Hopcroft, Wolfgang Paul, and Leslie Valiant. On time versus space. *Journal of the ACM*, 24(2):332–337, 1977. doi: 10.1145/322003.322015.
- John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Pearson, 2001.
- Neil Immerman. Nondeterministic space is closed under complementation. *SIAM Journal on Computing*, 17(5):935–938, 1988. doi: 10.1137/0217058.
- Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, The IBM Research Symposia Series, pages 85–103. Plenum Press, New York, 1972. doi: 10.1007/978-1-4684-2001-2_9. URL https://doi.org/10.1007/978-1-4684-2001-2_9. Associate editor: Jean D. Bohlinger.
- Jaegwon Kim. *Supervenience and Mind: Selected Philosophical Essays*. Cambridge University Press, Cambridge, 1993. doi: 10.1017/CBO9780511625220.
- A. N. Kolmogorov. Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1(1):1–7, 1965.
- James Ladyman and Don Ross. *Every Thing Must Go: Metaphysics Naturalized*. Oxford University Press, 2007.
- R. Landauer. Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development*, 5(3):183–191, 1961.
- Seth Lloyd. Ultimate physical limits to computation. *Nature*, 406(6799):1047–1054, 2000.
- David Marr. *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information*. W.H. Freeman, 1982.
- Allen Newell. The knowledge level. *Artificial Intelligence*, 18(1):87–127, 1982. doi: 10.1016/0004-3702(82)90012-1.
- Christos H. Papadimitriou. *Computational Complexity*. Addison-Wesley, 1994. ISBN 9780201530827. URL <https://books.google.com/books?id=JogZAQAAIAAJ>.

- Gualtiero Piccinini. *Physical Computation: A Mechanistic Account*. Oxford University Press, UK, 2015.
- Gualtiero Piccinini and Corey Maley. Computation in Physical Systems. In Edward N. Zalta, editor, *The Stanford Encyclopedia of Philosophy*. Stanford University, Stanford, Summer 2021 edition, 2021. URL <https://plato.stanford.edu/archives/sum2021/entries/computation-physicalsystems/>.
- Thomas W. Polger and Lawrence A. Shapiro. *The Multiple Realization Book*. Oxford University Press, Oxford, 2016. doi: 10.1093/acprof:oso/9780198732891.001.0001.
- Emil L. Post. Finite combinatory processes—formulation 1. *The Journal of Symbolic Logic*, 1(3):103–105, 1936. doi: 10.2307/2269031.
- Huw Price. *Time’s Arrow and Archimedes’ Point: New Directions for the Physics of Time*. Oxford University Press, New York, 1996.
- Hilary Putnam. *Representation and Reality*. MIT Press, Cambridge, MA, 1988.
- M. O. Rabin and D. Scott. Finite automata and their decision problems. *IBM Journal of Research and Development*, 3(2):114–125, 1959. doi: 10.1147/rd.32.0114. URL <https://doi.org/10.1147/rd.32.0114>.
- Jorma Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978. doi: 10.1016/0005-1098(78)90005-5. URL [https://doi.org/10.1016/0005-1098\(78\)90005-5](https://doi.org/10.1016/0005-1098(78)90005-5).
- Don Ross. Rainforest realism: A Dennettian theory of existence. In Don Ross, Andrew Brook, and David Thompson, editors, *Dennett’s Philosophy: A Comprehensive Assessment*, pages 147–168. MIT Press, Cambridge, MA, 2000.
- Carlo Rovelli. *The Order of Time*. Riverhead Books, New York, 2018. Translated by Erica Segre and Simon Carnell.
- Walter J. Savitch. Relationships between nondeterministic and deterministic tape complexities. *Journal of Computer and System Sciences*, 4(2):177–192, 1970. doi: 10.1016/S0022-0000(70)80006-X. URL [https://doi.org/10.1016/S0022-0000\(70\)80006-X](https://doi.org/10.1016/S0022-0000(70)80006-X).
- Oron Shagrir. Why we view the brain as a computer. *Synthese*, 153(3):393–416, 2006.
- Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27:379–423, 623–656, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x.
- Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 3rd edition, 2012. ISBN 9781133187790. URL <https://math.mit.edu/~sipser/book.html>.
- Lee Smolin. *Time Reborn: From the Crisis in Physics to the Future of the Universe*. Houghton Mifflin Harcourt, 2013.

- R.J. Solomonoff. A formal theory of inductive inference. part i. *Information and Control*, 7(1):1–22, 1964.
- Róbert Szelepcsényi. The method of forced enumeration for nondeterministic automata. *Acta Informatica*, 26(3):279–284, 1988. doi: 10.1007/BF00299636.
- A. M. Turing. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1):230–265, 1936.
- Peter van Emde Boas. Machine models and simulations. In Jan van Leeuwen, editor, *Handbook of Theoretical Computer Science, Volume A: Algorithms and Complexity*, pages 1–66. Elsevier, Amsterdam, 1990.
- David Wallace. Everett and structure. *Studies in History and Philosophy of Science Part B: Studies in History and Philosophy of Modern Physics*, 34(1):87–105, 2003. doi: 10.1016/S1355-2198(02)00085-0.
- John Archibald Wheeler. Information, physics, quantum: The search for links. In Wojciech H. Zurek, editor, *Complexity, Entropy, and the Physics of Information*, pages 3–28. Addison-Wesley, Redwood City, CA, 1990.
- Ryan Williams. Simulating time with square-root space. 2025. Preprint, arXiv:2502.17779.